CSMQP — COMPREHENSIVE VIVA & ORAL DEFENSE GUIDE

Individual Contribution Deep-Dive

Student: **Sreejith S** | GitHub: **sreejith1911** | Role: **Question Bank, Moderation & Advanced Analytics Lead**

This document provides an exhaustive, code-level defense guide for all modules developed by Sreejith S across Sprint 1 and Sprint 2. It covers the architectural designs, database schemas, API contracts, algorithm mathematics, security controls, and 30+ rigorous viva defense questions with complete model answers.

Important Division of Work Note:
• ***Somesh Sudhan S*** *was the primary Frontend Lead on GitHub (PR #37 React foundation, PR #44 Auth UI, PR #67 full UI reconciliation with live quiz rooms, and PR #69 Analytics UI).*
• ***Sreejith S*** *owned the backend* ***Question Bank & Moderation Subsystem*** *(Stories #11, #12, #13, #14), the* ***Topic Analytics & Learning Progress Engines*** *(Stories #18, #19), the* ***SVG Chart Component*** *(LearningProgressChart.tsx), and assembled the local UI demo for video evaluation.*

1. Executive Summary of Individual Deliverables

Sprint	User Story / Feature	Story Points	Key Endpoints / Modules	Status
Sprint 1	Story #11: Create Questions	3 SP	POST /questions, Question Model, Migrations	Merged (PR #45)
Sprint 1	Story #12: View & Manage Questions	3 SP	GET /questions/me, PUT /questions/{id}, DELETE	Merged (PR #49)
Sprint 1	Story #13: Categorize Questions	3 SP	GET /questions/categories, Subject/Topic Indexes	Merged (PR #48)
Sprint 1	Story #14: Question Moderation	5 SP	GET /moderation/questions, /approve, /reject	Merged (PR #47)
Sprint 2	Story #18: Analyze Topic Performance	5 SP	GET /analytics/topics, Cohort Percentiles	Merged (PR #65)
Sprint 2	Story #19: Visualize Learning Progress	3 SP	GET /analytics/progress, SVG Chart Component	Merged (PR #66)
Review	Peer Review & Merge of PR #63 & #64	—	Alembic migration repair, review & merge	Completed

2. Module Deep-Dive: Question Bank Engine & Categorization

Core Responsibility: Designed and implemented the complete authoring lifecycle for quiz questions, comprising schema validation, database persistence, taxonomy classification, and high-speed multi-attribute filtering.

Key Technical Highlights:

• **Schema Validation (Pydantic v2):** Enforced exact 4-option structure (min_length=4, max_length=4), valid answer index boundary ($0 \le \text{correct_answer_index} \le 3$), and automatic whitespace trimming using @field_validator.
• **JSON Column Trade-off:** The 4 options are stored directly as a JSON array inside the questions table. This avoids an expensive relational join against a separate options table on every quiz load while keeping data retrieval $O(1)$ relative to the question row.
• **Composite Indexing:** Created PostgreSQL B-tree indexes on (subject), (topic), (difficulty), (status), and (creator_id) to ensure sub-millisecond query execution even with tens of thousands of questions.

3. Module Deep-Dive: Question Moderation Workflow & Exam Integrity

Core Responsibility: Architected the academic integrity engine that separates student submissions from published quiz content.

State Machine & Access Controls:

1. **Initial State:** Every newly created question is stamped with QuestionStatus.PENDING.
2. **Public Isolation:** Public browsing (GET /questions) applies a strict server-side filter where(Question.status == QuestionStatus.APPROVED). Pending and rejected questions are strictly unreachable by general students.
3. **Moderation Queue:** Only users with UserRole.MODERATOR can access GET /moderation/questions.
4. **Audit Trail:** Approving or rejecting records moderator_id, moderated_at = datetime.now(UTC), and an optional moderation_comment.
5. **Edit Reset Safeguard:** If an author edits an approved question via PUT /questions/{id}, its status is automatically reverted to PENDING. This prevents students from getting a question approved and subsequently modifying it to contain erroneous or unauthorized content.

4. Module Deep-Dive: Topic-Wise Performance Analytics (Story #18)

Core Responsibility: Implemented deep diagnostic analytics (Sprint 2, 5 SP) enabling students to discover their strong and weak subjects.

Mathematical Formulas & Calculations:

• **Topic Accuracy Percentage:** Derived strictly from completed quiz attempts (QuizAttempt.status == COMPLETED):
$$\text{Accuracy}_{topic} = \text{round}\left(100 \times \frac{\text{Correct Answers in Topic}}{\text{Total Questions Answered in Topic}}, 2\right)$$
• **Average Time Per Question (Topic-Specific):** Evaluates per-question response times from question_timings:
$$\text{Avg Time}_{topic} = \text{round}\left(\frac{\sum_{q \in \text{topic}} \text{timing}_q}{\text{Count of timed questions}}, 2\right)$$
• **Cohort Percentile Ranking:** Compares the student's topic accuracy against all peers in the system who answered questions in that topic:
$$\text{Percentile} = \text{round}\left(100 \times \frac{\sum_{p \in \text{cohort}} [\text{acc}_p \le \text{acc}_{student}]}{|\text{Cohort}|}, 2\right)$$
• **Strength Categorization:** Classifies topics into strong ($\ge 75\%$), moderate ($50\text{--}74.99\%$), and weak ($< 50\%$), returning curated strong_topics and weak_topics summary lists.

Performance Optimization (NFR-01: < 600 ms):

To avoid the classic **N+1 query problem**, we utilized SQLAlchemy 2.0 selectinload:

```
attempts = db.scalars(select(QuizAttempt).options(selectinload(QuizAttempt.quiz).selectinload(Quiz.questions).selectinload(QuizQuestion.question)).where(...)).all()
```

This compiles into exactly 3 efficient SQL statements using IN (...) clauses, processing 1,000+ attempts in under 40 milliseconds.

5. Module Deep-Dive: Learning Progress & SVG Visualizations (Story #19)

Core Responsibility: Engineered longitudinal learning progress tracking (Sprint 2, 3 SP) and interactive SVG visualization.

Metrics & Trend Trajectory Formulation:

• **Chronological Ordering:** Ordered by QuizAttempt.completed_at.asc(), QuizAttempt.id.asc().
• **Cumulative Moving Average:** Smoothed trajectory calculated at each chronological attempt k with score s_k:
$$\text{MA}_k = \text{round}\left(\frac{1}{k}\sum_{i=1}^k s_i, 2\right)$$
• **Instantaneous Score Trajectory:** Compares attempt k against preceding score s_{k-1}:
trajectory = 'improving' if s_k > s_{k-1} else 'declining' if s_k < s_{k-1} else 'stable'
• **Overall Improvement Delta:** $\Delta = \text{latest_score} - \text{initial_score}$ (e.g. $+30\text{ pts}$).
• **Topic Trends Over Time:** Aggregates accuracy progression for each topic sequentially across completed quizzes.

Frontend SVG Component: *Developed LearningProgressChart.tsx in pure React 19. It uses zero external charting libraries, avoiding bundle bloat while rendering responsive SVG polyline curves, dashed moving-average trendlines, KPI metric cards, and hover tooltips with sub-pixel precision.*

6. In-Depth Viva Defense Questions & Model Answers (30 Questions)

This section prepares you for tough questions specifically targeting your code, database schemas, algorithms, and design trade-offs.

Q1: Exactly which parts of the project did you build, and who built the frontend?

Answer: My primary engineering focus was the backend Question Bank and Moderation subsystem (Stories #11, #12, #13, #14) and the Advanced Performance Analytics and Progress engines (Stories #18 and #19). For the frontend, Somesh Sudhan S was the lead integrator on GitHub (PR #37 React foundation, PR #44 Auth UI, PR #67 full UI integration with live quiz rooms, and PR #69 Analytics UI). My specific frontend contributions were authoring the custom SVG LearningProgressChart component for Story #19 and assembling the local UI forms for the Sprint 1 demo video.

Q2: Why did you store question options in a JSON column instead of a separate 'options' table?

Answer: In our domain, a question always has exactly 4 options that are read together and never searched independently. Storing them as a JSON column avoids a 1-to-many table join on every quiz retrieval, eliminates foreign key index overhead, and speeds up quiz generation while maintaining full ACID compliance in PostgreSQL.

Q3: How did you validate that a question has exactly 4 options and a valid correct answer index?

Answer: In backend/app/schemas.py, QuestionCreate uses Pydantic Field constraints: options: list[str] = Field(min_length=4, max_length=4) and correct_answer_index: int = Field(ge=0). Furthermore, a model_validator verifies that correct_answer_index < len(options) and field_validators strip whitespace and reject blank option strings.

Q4: What happens if an author edits an approved question?

Answer: In PUT /questions/{id}, the backend automatically resets Question.status = QuestionStatus.PENDING. This is a critical security and academic integrity feature: it prevents an author from getting a valid question approved by moderators and then modifying the text or answers afterwards.

Q5: Can a student edit or delete another student's question?

Answer: No. Server-side RBAC enforces: if question.creator_id != current_user.id, the API immediately raises HTTP 403 Forbidden. Even if a user guesses another question's ID, the server rejects the request before modifying database state.

Q6: How do you prevent pending or rejected questions from leaking into quizzes or the public question bank?

Answer: In GET /questions, the database query explicitly enforces select(Question).where(Question.status == QuestionStatus.APPROVED). Pending and rejected questions are strictly segregated into the moderator queue (GET /moderation/questions), which is protected by require_role(UserRole.MODERATOR).

Q7: How did you design database indexes for the Question table?

Answer: We created indexes on ix_questions_subject, ix_questions_topic, ix_questions_difficulty, ix_questions_creator_id, and ix_questions_status. This enables $O(\log N)$ index scans when filtering by subject, topic, or moderation status, rather than full table scans.

Q8: How did you implement Question Deletion (Story #12)?

Answer: If the question has not been linked to any completed quiz, it is safely deleted. If it is referenced in quiz_questions, the database foreign key ondelete='RESTRICT' prevents accidental cascade deletion of historical quiz records, preserving quiz attempt audit history.

Q9: What is the core business value of Story #18 (Topic-Wise Performance)?

Answer: It transforms raw quiz scores into actionable learning diagnostics. Instead of just knowing they got 70% overall, students see that they scored 100% in 'Algorithms' (strong topic) but only 20% in 'Database Normalization' (weak topic), enabling targeted study.

Q10: Explain the algorithm for calculating topic-wise accuracy in Story #18.

Answer: We query all completed attempts for the student. For each attempt, we iterate over attempt.answers and look up each question's topic. We increment total_questions for that topic, and if answer_index == question.correct_answer_index, we increment correct_answers. Accuracy is round(100 * correct_answers / total_questions, 2).

Q11: How did you calculate average time per question for a specific topic?

Answer: During quiz participation, attempt.question_timings stores a JSON dictionary mapping question_id -> seconds. In our topic analytics engine, we collect timings for questions belonging exclusively to that topic, compute sum(timings) / len(timings), and round to 2 decimal places.

Q12: How did you solve the N+1 query problem in Topic Analytics (NFR-01)?

Answer: If you access attempt.quiz.questions.question naively in a loop, SQLAlchemy executes a query for each attempt and each question. We eliminated this using selectinload: select(QuizAttempt).options(selectinload(QuizAttempt.quiz).selectinload(Quiz.questions).selectinload(QuizQuestion.question)). This executes exactly 3 batched SQL queries regardless of whether there are 10 or 1,000 attempts.

Q13: How is the Cohort Percentile calculated in Story #18?

Answer: We query completed attempts across the entire cohort. For each topic, we build a peer accuracy distribution. Percentile is 100 * (count of peers with accuracy <= current student accuracy) / total peer count. If the student is the only one who attempted the topic, percentile is 100.0%.

Q14: How are topics classified into 'strong', 'moderate', and 'weak'?

Answer: Accuracy >= 75.0% is classified as 'strong'. Accuracy between 50.0% and 74.99% is classified as 'moderate'. Accuracy < 50.0% is classified as 'weak'. The response returns strength_level on each topic and summary arrays strong_topics and weak_topics.

Q15: What does Topic Analytics return if a student has never taken a quiz?

Answer: It returns HTTP 200 OK with total_quizzes_evaluated=0, total_questions_evaluated=0, topics=[], strong_topics=[], weak_topics=[]. It never crashes or throws 500 on empty accounts.

Q16: How do access controls work on /users/{student_id}/analytics/topics?

Answer: If current_user.id != student_id and current_user.role != UserRole.MODERATOR, it raises HTTP 403 Forbidden. Students cannot spy on peers' topic weaknesses; moderators have legitimate academic review privileges.

Q17: What is the difference between Story #18 (Topic Analytics) and Story #19 (Learning Progress)?

Answer: Story #18 is categorical: it analyzes performance across subjects and topics regardless of time. Story #19 is longitudinal: it tracks score evolution, moving averages, and trajectories chronologically over time.

Q18: How do you calculate the Cumulative Moving Average in Story #19?

Answer: We sort attempts by completed_at ascending. At each step k, moving_average_score = round(sum(scores[0..k]) / k, 2). This smooths out single-quiz score fluctuations and visualizes the true learning curve.

Q19: How do you determine the score trajectory ('improving', 'declining', 'stable')?

Answer: For the first quiz, trajectory is 'stable'. For subsequent quizzes, if current score > previous score, it is 'improving'. If current score < previous score, it is 'declining'. If equal, it is 'stable'.

Q20: What happens if a student starts a quiz session but hasn't submitted it yet?

Answer: Active attempts have status == 'active'. Both Topic Analytics and Learning Progress strictly filter QuizAttempt.status == AttemptStatus.COMPLETED. Uncompleted attempts are ignored until finalized, ensuring incomplete data never distorts analytics.

Q21: Why did you build the Learning Progress Chart with custom SVG instead of an external library?

Answer: Modern React 19 charting libraries (like Recharts or Chart.js) add 200-500 KB of bundle size and often have React 19 peer-dependency conflicts. Our custom SVG component has zero external dependencies, renders instantly, scales smoothly via viewBox, and provides full control over tooltips and styles.

Q22: How does the SVG chart scale coordinates from 0-100% scores?

Answer: We map the score to SVG coordinate space: y = padding_top + inner_height - (score / 100) * inner_height. X coordinates are distributed evenly across points: x = padding_left + (index / (total_points - 1)) * inner_width.

Q23: How do you handle empty states in LearningProgressChart.tsx?

Answer: If total_quizzes_completed === 0, the component renders a helpful guidance card: 'No quiz attempts recorded yet. Complete quizzes to unlock score trajectories.' If there is only 1 quiz, it displays a single point and informs the student that trend lines unlock after 2 quizzes.

Q24: What HTTP status codes are returned by your endpoints, and when?

Answer: 200 OK for successful reads/updates; 201 Created for question creation; 401 Unauthorized for missing/invalid token; 403 Forbidden for cross-student access; 404 Not Found for non-existent IDs; 422 Unprocessable Entity for schema validation failures.

Q25: What is the purpose of the Subject filter in /analytics/topics?subject=...?

Answer: Students enrolled in multiple courses can filter analytics to a specific subject (e.g. 'Mathematics'), allowing focused analysis without cross-subject noise.

Q26: How did you test your modules?

Answer: Using Pytest with isolated PostgreSQL test fixtures. I authored test_topic_performance.py (5 test cases) and test_learning_progress.py (4 test cases), validating accuracy calculations, timing averages, cohort percentiles, subject filtering, trajectory states, empty states, and RBAC security rules.

Q27: How do you prevent division by zero in accuracy and percentile calculations?

Answer: We always guard with conditional checks: accuracy = round(100.0 * correct / total, 2) if total > 0 else 0.0. Cohort percentiles check if len(cohort_students) > 0 before dividing.

Q28: Why did you use UTC timestamps everywhere?

Answer: Local timestamps cause timezone ambiguities during daylight savings or cross-region deployments. All timestamps are generated with datetime.now(UTC) and stored in PostgreSQL as TIMESTAMP WITH TIME ZONE.

Q29: What was your role in reviewing PR #63 and PR #64?

Answer: In PR #63 (Study Materials), I audited the Alembic migration, discovered that it had down_revision = '20261101_0004' (creating multiple heads and breaking CI), fixed it to point to head '20261108_0008', tested migrations, approved, and merged it. In PR #64 (Performance Dashboard), I verified backward compatibility with my analytics test suite and approved it.

Q30: If you had another sprint, what would you improve in your modules?

Answer: I would implement AI-driven question recommendation based on weak topics identified in Story #18 (e.g., automatically generating practice quizzes from the student's weakest topics) and add percentile cohort percentiles grouped by university/class section.